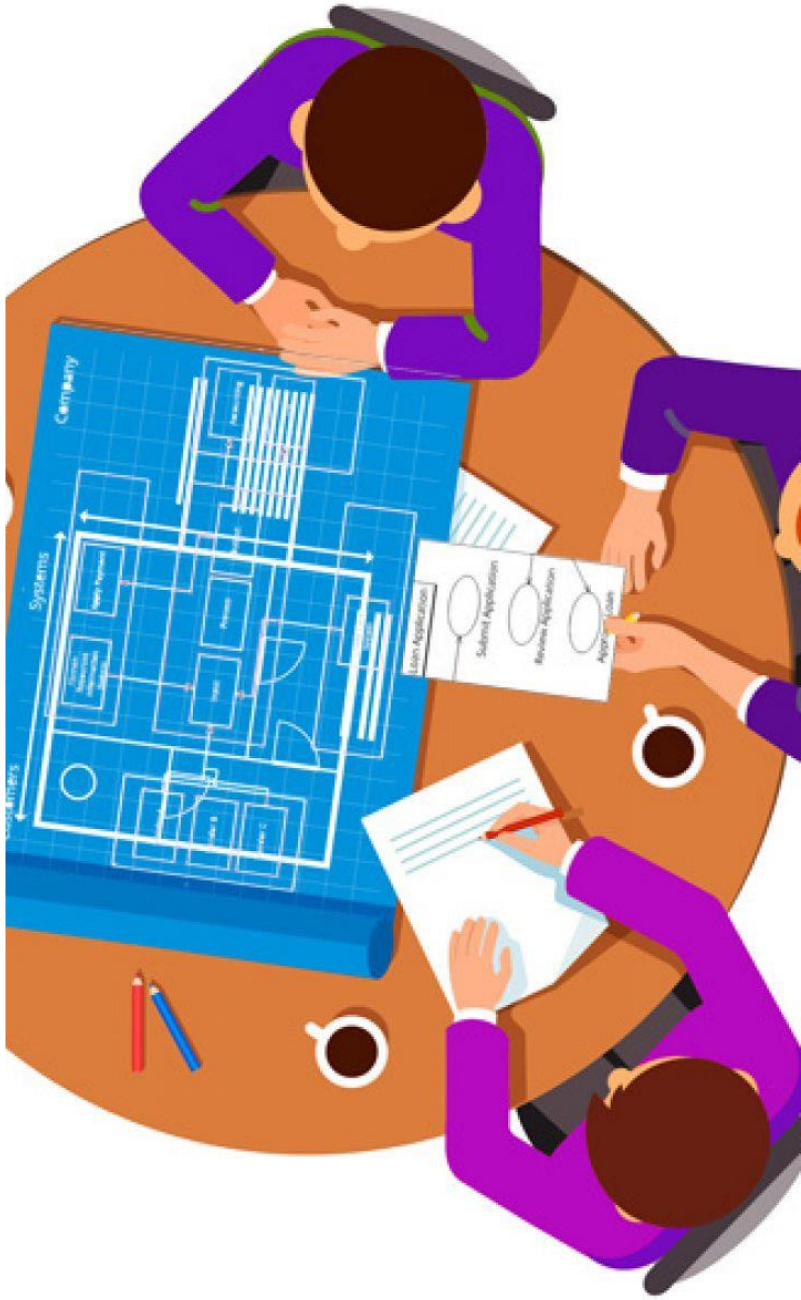
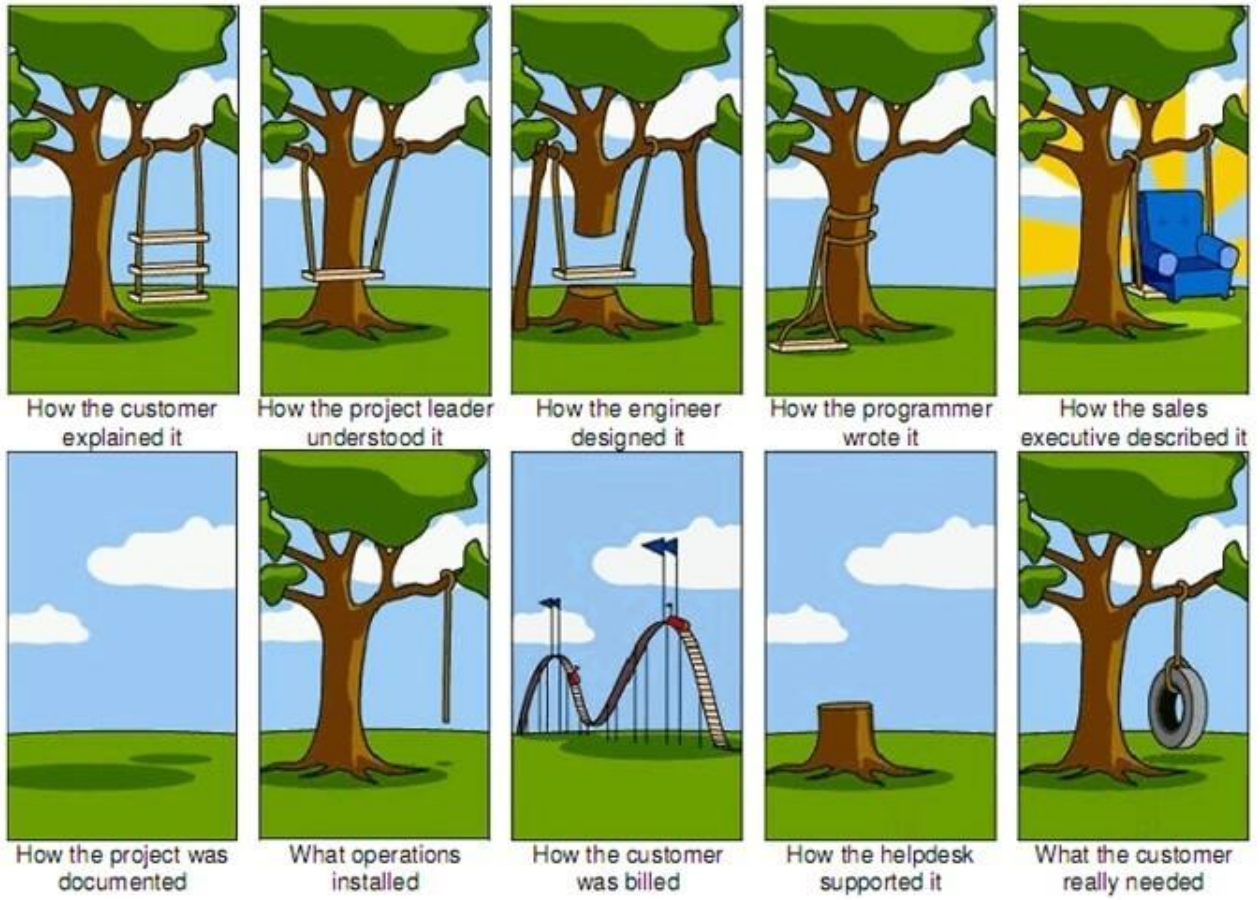
System Analysis Stage

Requirement Gathering



What happens if you skip gathering requirements for your software project?

Depending on your project methodology, you may do this step at the **beginning during a Discovery phase**, you may do it during the project **within each sprint or build cycle**, or you may skip it altogether and **hope for the best**. That **last option** is a simple way to sabotage your project and guarantee a lot of late nights and awkward status meetings.



Outline

- Introduction
- Stakeholder analysis
- Artefact-driven elicitation techniques
  - Background study
  - Questionnaires
  - Storyboards
  - Scenarios
- Stakeholder-driven elicitation techniques
  - Interviews
  - Observation & ethnographic studies
- Requirements validation
- Requirements Prioritization

Requirements engineering

The process of **establishing the services** that the **customer requires from a system and the constraints** under which it operates and is developed.

The **requirements** themselves are the **descriptions of the system services and constraints** that are generated during the requirements engineering process.

Requirements abstraction

- “If a company wishes to let a contract for a large software development project, it must define its needs in a sufficiently abstract way that a solution is not pre-defined. The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization’s needs. Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do. Both documents may be called the requirements document for the system.”

Requirements specification

The process of writing down the user and system requirements in a requirements document.

User requirements have to be understandable by end-users and customers who do not have a technical background.

System requirements are more detailed requirements and may include more technical information.

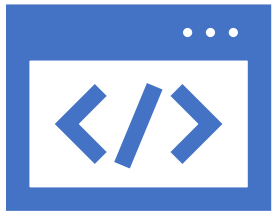
The requirements may be part of a contract for the system development

It is therefore important that these are as complete as possible.

Ways of writing a system requirements specification

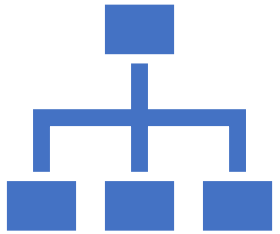
Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Types of requirement



User requirements

Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.



System requirements

A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

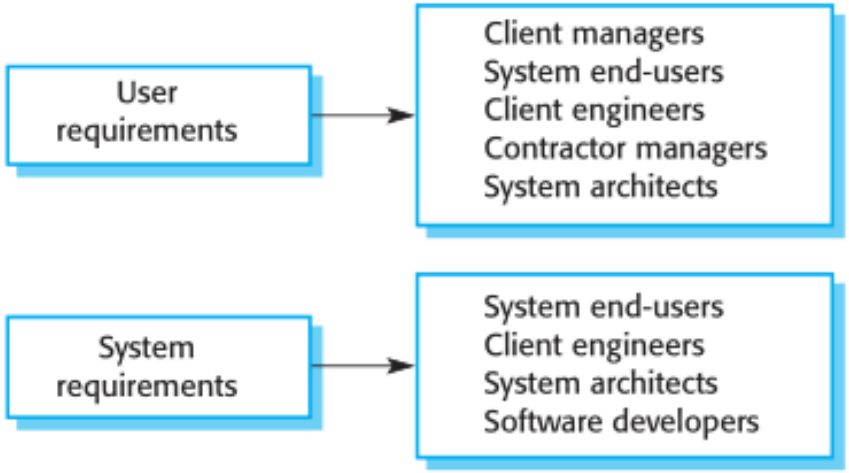
User and system requirements

User requirements definition

1. The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
1.2 The system shall generate the report for printing after 17.30 on the last working day of the month.
1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
1.4 If drugs are available in different dose units (e.g. 10mg, 20mg, etc.) separate reports shall be created for each dose unit.
1.5 Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.



Readers of different types of requirements specification

Functional and non-functional requirements

- Functional requirements
  - Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
  - May state what the system should not do.

- Non-functional requirements
  - Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
  - Often apply to the system as a whole rather than individual features or services.

- Domain requirements
  - Constraints on the system from the domain of operation

Functional requirements

- Describe functionality or system services.
- Depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

Functional requirement examples for a Mental Care System

- A user shall be able to search the appointments lists for all clinics.
- The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
- Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.

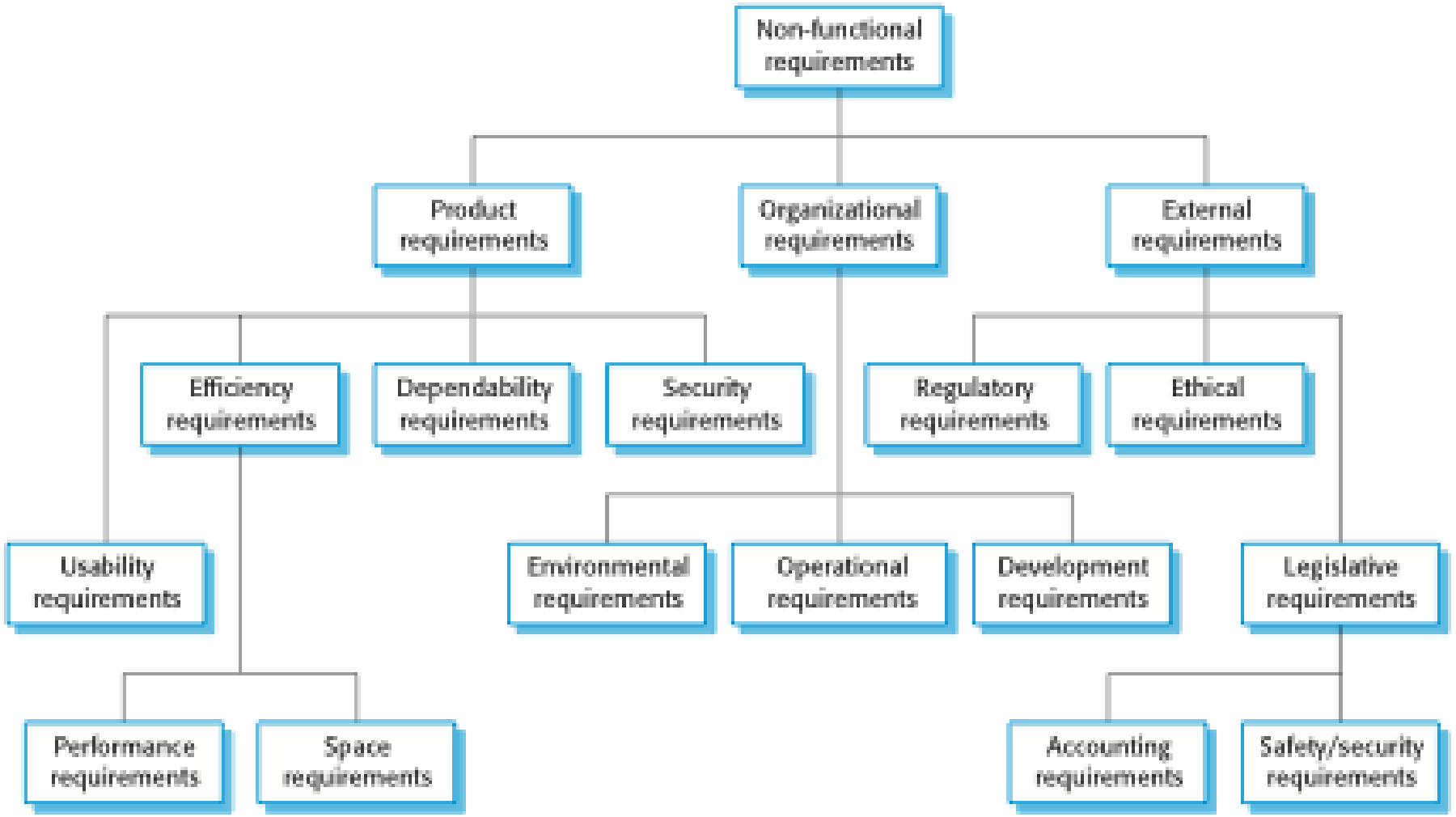
Non-functional requirements

- These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.

- Process requirements may also be specified mandating a particular IDE, programming language or development method.

- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.

Types of non-functional requirement



Non-functional requirements implementation

- Non-functional requirements may affect the overall architecture of a system rather than the individual components.
  - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.

- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
  - It may also generate requirements that restrict existing requirements.

Outline

- Introduction
- Stakeholder analysis
- Artefact-driven elicitation techniques
  - Background study
  - Questionnaires
  - Storyboards
  - Scenarios
- Stakeholder-driven elicitation techniques
  - Interviews
  - Observation & ethnographic studies
- Requirements validation
- Requirements Prioritization

Stakeholder analysis

- Stakeholder cooperation is essential for successful req. gathering
  - Elicitation = cooperative learning
- Representative sample must be selected to ensure adequate, comprehensive coverage of the problem world
  - dynamic selection as new knowledge is acquired
- Selection based on ...
  - relevant position in the organization
  - role in making decisions, reaching agreement
  - type of contributed knowledge, level of domain expertise
  - exposure to perceived problems
  - personal interests, potential conflicts
  - influence in system acceptance
- **Knowledge acquisition from stakeholders is difficult**: Distributed sources, Different background, Personnel turnover.

Background study

- Collect, read, synthesize documents about...
  - the **organization**: organizational charts, business plans, financial reports, meeting minutes, etc
  - the **domain**: books, surveys, articles, regulations, reports on similar systems in the same domain
  - the **system-as-is**: documented workflows, procedures, business rules; exchanged documents; defect/complaint reports, change requests, etc.
- Provides basics for getting prepared before meeting stakeholders

  => prerequisite to other techniques

The current system is called the **as-is system**
The new system is called the **to-be system**

Questionnaires

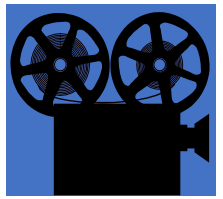
- Submit a list of questions to selected stakeholders, each with a list of possible answers (+ brief context if needed)
  - **Multiple choice** question: one answer to be selected from answer list
  - **Weighting** question: list of statements to be weighted...
    - qualitatively ('high', 'low", ...), or
    - quantitatively (percentages)
    - to express perceived importance, preference, risk etc.
- Effective for acquiring subjective info quickly, cheaply, remotely from many people
- Helpful for preparing better focused interviews (see next)

Questionnaires should be carefully prepared

=> Guidelines for questionnaire design/validation:

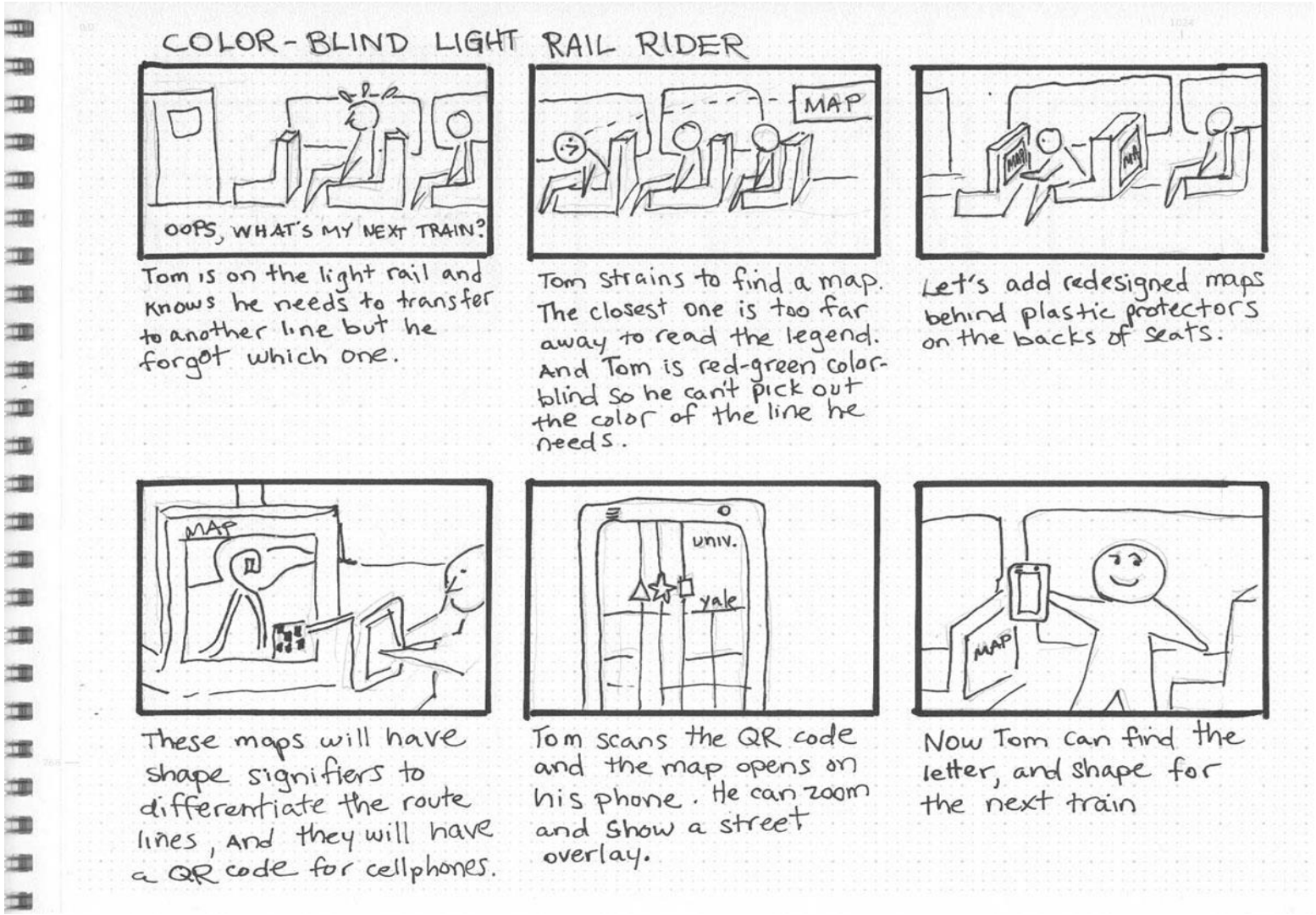
- Select a representative, statistically significant sample of people; provide motivation for responding
- Check coverage of questions, of possible answers
- Make sure questions, answers, formulations are unbiased & unambiguous
- Add implicitly redundant questions to detect inconsistent answers
- Have your questionnaire checked by a third party

Storyboards

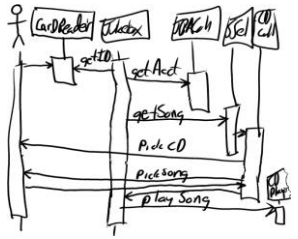


- **Goal**: acquire or validate info from concrete examples through narratives ...
  - how things are running in the system-*as-is*
  - how things should be running in the system-*to-be*
- **Storyboard**: tells a story by a sequence of snapshots
  - Snapshot = sentence, sketch, slide, picture, etc.
  - Possibly structured with annotations:
    - WHO are the players, WHAT happens to them, WHY this happens, WHAT IF this does / does *not* happen, etc
  - **Passive** mode (for validation): stakeholders are told the story
  - **Active** mode (for joint exploration): stakeholders contribute

Storyboards: Color Blind Example



Scenarios



- Illustrate typical sequences of interaction among system components to meet an implicit objective
- Widely used for...
  - **explanation** of system-*as-is*
  - **exploration** of system-*to-be* + elicitation of further info ...

    e.g. WHY this interaction sequence ?

    WHY among these components ?
  - specification of acceptance test cases
- Represented by text or diagram

Types of scenario

- **Positive** scenario = one behavior the system should cover (example)
- **Negative** scenario = one behavior the system should exclude (counter-example)

- **Normal** scenario: everything proceeds as expected
- **Abnormal** scenario = a desired interaction sequence in exception situation (still positive)

  e.g. meeting initiator not authorized
  participant constraints not valid

Outline

- Introduction
- Stakeholder analysis
- Artefact-driven elicitation techniques
  - Background study
  - Questionnaires
  - Storyboards
  - Scenarios
- Stakeholder-driven elicitation techniques
  - Interviews
  - Observation & ethnographic studies
- Requirements validation
- Requirements Prioritization

Interviews

- Primary technique for knowledge elicitation
  1. Select stakeholder specifically for info to be acquired
     (domain expert, manager, salesperson, end-user, consultant, ...)
  2. Organize meeting with interviewee, ask questions, record answers
  3. Write report from interview transcripts
  4. Submit report to interviewee for validation & refinement
- Single interview may involve multiple stakeholders
  - ☺ saves times
  - ☹ weaker contact; individuals less involved, speak less freely
- Interview **effectiveness** (measured as weighted ratio):
  (*utility* x *coverage* of acquired info) / time needed

Types of interview

- **Structured** interview: predetermined set of questions
  - specific to purpose of interview
  - some open-ended, others with pre-determined answer set

  => more focused discussion, no rambling among topics
- **Unstructured** interview: no predetermined set of questions
  - free discussion about system-as-is, perceived problems, proposed solutions

  => exploration of possibly overlooked issues

=> Effective interviews should mix both modes ...
  - start with structured parts
  - shift to unstructured parts as felt necessary

Interviews: strengths & difficulties

☺ May reveal info not acquired through other techniques
- how things are running *really*, personal complaints, suggestions for improvement, ...

☺ On-the-fly acquisition of info appearing relevant
- new questions triggered from previous answers

☹ Acquired info might be subjective (hard to assess)

☹ Potential inconsistencies between different interviewees

☹ Effectiveness critically relies on interviewer's attitude, appropriateness of questions

=> *Interviewing guidelines*

Guidelines for effective interviews

- Identify the right interviewee sample for full coverage of issues
  - different responsibilities, expertise, tasks, exposure to problems
- Come prepared, to focus on right issue at right time
  - backgound study first
  - predesign a sequence of questions for **this** interviewee
- Centre the interview on the interviewee's work & concerns
- Keep control over the interview
- Make the interviewee feel comfortable
  - *Start:* break ice, provide motivation, ask easy questions
  - Consider the person too, not only the role
  - Do always appear as a trustworthy partner
- Be focused, open-minded flexible in case of unexpected answers
- keep open-ended questions for the end
- Ask *why*-questions without being offending
- Edit & structure interview transcripts while still fresh in mind
- Keep interviewee in the loop

Observation & ethnographic studies

- Focus on **task** elicitation in the system-as-is
- Understanding a task is often easier by observing people performing it (rather than verbal or textual explanation)
- **Passive** observation: no interference with task performers
  - Watch from outside, record (notes, video), edit transcripts, interpret
  - **Protocol analysis**: task performers concurrently explain it
  - **Ethnographic studies**: over long periods of time, try to discover emergent properties of social group involved
    - about task performance + attitudes, reactions, gestures, …
- **Active** observation: you get involved in the task, even become a team member

Outline

- Introduction
- Stakeholder analysis
- Artefact-driven elicitation techniques
  - Background study
  - Questionnaires
  - Storyboards
  - Scenarios
- Stakeholder-driven elicitation techniques
  - Interviews
  - Observation & ethnographic studies
- Requirements validation
- Requirements Prioritization

Requirements Validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error **costs are high** so validation is very important
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.
- Requirements checking, includes:
  - Validity. Does the system provide the functions which best support the customer's needs?
  - Consistency. Are there any requirements conflicts?
  - Completeness. Are all functions required by the customer included?
  - Realism. Can the requirements be implemented given available budget and technology
  - Verifiability. Can the requirements be checked?

Requirements validation techniques

- Requirements reviews
  - Systematic manual analysis of the requirements.
    - **Verifiability**: Is the requirement realistically testable?
    - **Comprehensibility**: Is the requirement properly understood?
    - **Traceability**: Is the origin of the requirement clearly stated?
    - **Adaptability**: Can the requirement be changed without a large impact on other requirements?
- Prototyping
  - Using an executable model of the system to check requirements.
- Test-case generation
  - Developing tests for requirements to check testability.

Requirement Prioritization

- Prioritization can be applied to requirements/User Stories, tasks, products, use cases, acceptance criteria and tests.
- **MoSCoW** prioritization, also known as the MoSCoW method or MoSCoW analysis, is a popular prioritization technique for managing requirements.
- The acronym MoSCoW represents four categories of initiatives:
  - **Must-have**:
    Non-negotiable product needs that are mandatory for the team.
  - **Should-have**:
    Important initiatives that are not vital but add significant value.
  - **Could-have**:
    Nice to have initiatives that will have a small impact if left out.
  - **Will not have right now**:
    Initiatives that are not priority for this specific time frame.
- Categorizing a requirement as a Should Have or Could Have does not mean it won't be delivered; simply that delivery is not guaranteed.

Must Have

- These provide the Minimum Usable SubseT (MUST) of requirements which the project guarantees to deliver. These may be defined using some of the following:
  - No point in delivering on target date without this; if it were not delivered, there would be no point deploying the solution on the intended date
  - Not legal without it
  - Unsafe without it
  - Cannot deliver a viable solution without it

- Ask the question ‘what happens if this requirement is not met?’ If the answer is ‘cancel the project – there is no point in implementing a solution that does not meet this requirement’, then it is a Must Have requirement.

Should & Could Have

- **Should Have** requirements are defined as:
  - Important but not vital
  - May be painful to leave out, but the solution is still viable
  - May need some kind of workaround, e.g. management of expectations, some inefficiency, an existing solution, paperwork etc. The workaround may be just a temporary one

- **Could Have** requirements are defined as:
  - Wanted or desirable but less important
  - Less impact if left out (compared with a Should Have)

- One way of **differentiating** a Should Have requirement from a Could Have is by reviewing the degree of pain caused by the requirement not being met, measured in terms of business value or numbers of people affected.

Won’t Have

- These are requirements which the project team has agreed will not be delivered (as part of this timeframe).
- They are recorded in the Prioritized Requirements List where they help clarify the scope of the project.
- This avoids them being informally reintroduced at a later date.
- This also helps to manage expectations that some requirements will simply not make it into the Deployed Solution, at least not this time around

References

- Requirements Engineering: From System Goals to UML Models to Software Specifications
- https://www.productplan.com/glossary/moscow-prioritization
- https://www.agilebusiness.org/dsdm-project-framework/moscow-prioririsation.html